

Sql

Function	Purpose	Example	Output
<code>LENGTH()</code>	Characters count	<code>LENGTH('Tarun')</code>	5
<code>CONCAT()</code>	Strings jodna	<code>CONCAT(name,city)</code>	TarunIndore
<code>SUBSTRING()</code>	Beech ka part	<code>SUBSTRING('Tarun',2,3)</code>	aru
<code>REPLACE()</code>	Text replace	<code>REPLACE('Tarun','a','@')</code>	T@run
<code>TRIM()</code>	Extra spaces remove	<code>TRIM(' Tarun ')</code>	Tarun
<code>LEFT()</code>	Left se characters	<code>LEFT('Tarun',2)</code>	Ta
<code>RIGHT()</code>	Right se characters	<code>RIGHT('Tarun',2)</code>	un
<code>REVERSE()</code>	String ulta	<code>REVERSE('Tarun')</code>	nuraT

</> SQL

-- 1. Name length greater than 5

```
SELECT * FROM student  
WHERE LENGTH(name)>5;
```

-- 2. Longest name

```
SELECT * FROM student  
ORDER BY LENGTH(name) DESC  
LIMIT 1;
```

-- 3. Display "Name - City"

```
SELECT CONCAT(name, ' - ',city)  
FROM student;
```

-- 4. First 3 letters

```
SELECT LEFT(name,3)  
FROM student;
```

-- 5. Last 2 letters

```
SELECT RIGHT(name,2)  
FROM student;
```

1. LTRIM()

Purpose

Left side ke spaces remove karta hai.

```
</> SQL
```

```
SELECT LTRIM('    Tarun');
```

Output

```
Tarun
```

Table Example

```
</> SQL
```

```
SELECT  
name,  
LTRIM(name)  
FROM student;
```

2. RTRIM()

Purpose

Right side ke spaces remove karta hai.

 SQL

```
SELECT RTRIM('Tarun   ');
```

Output

Tarun

Table Example

 SQL

```
SELECT  
name,  
RTRIM(name)  
FROM student;
```

3. MID() / SUBSTRING() ★ ★ ★

MID() MySQL me SUBSTRING() ka alias hai.

SQL

```
SELECT MID('Tarun Patidar',1,5);
```

Output

Tarun

Table Example

SQL

```
SELECT  
name,  
MID(name,1,3) AS short_name  
FROM student;
```

4. LOCATE() ★ ★ ★ ★

Purpose

String ke andar kisi character ya word ki position batata hai.

Syntax

⌞ SQL

```
LOCATE(search_string,string)
```

Example

⌞ SQL

```
SELECT LOCATE('a','Tarun');
```

5. ROW_NUMBER()

Unique ranking.

```
</> SQL
```

```
SELECT
```

```
name,
```

```
marks,
```

```
ROW_NUMBER()
```

```
OVER(
```

```
ORDER BY marks DESC
```

```
) rank_no
```

```
FROM student;
```

Department Wise

SQL

SELECT

name,

dept_id,

ROW_NUMBER()

OVER(

PARTITION BY dept_id

ORDER BY marks DESC

)

FROM student;

6. OVER() ★ ★ ★ ★ ★

Window define karta hai.

```
SQL
```

```
SELECT
```

```
name,
```

```
marks,
```

```
AVG(marks)
```

```
OVER()
```

```
FROM student;
```

Department Wise

SQL

SELECT

name,

dept_id,

AVG(marks)

OVER(

PARTITION BY dept_id

)

FROM student;

Even IDs

</> SQL

SELECT *

FROM student

WHERE MOD(id,2)=0;

Odd IDs

</> SQL

SELECT *

FROM student

WHERE MOD(id,2)=1;

8. LEAD()

Next row ki value.

 SQL

SELECT

name,

marks,

LEAD(marks)

OVER(

ORDER BY marks DESC

)

AS next_marks

FROM student;

9. DENSE_RANK() ★ ★ ★

Equal rank

Gap nahi.

SQL

SELECT

name,

marks,

DENSE_RANK()

OVER(

ORDER BY marks DESC

)

FROM student;

5ee-a57b-0a002300011a

10. DISTINCT

Duplicate remove karta hai.

SQL

```
SELECT DISTINCT city  
  
FROM student;
```

Multiple Columns

SQL

```
SELECT DISTINCT  
  
city,  
  
dept_id  
  
FROM student;
```

11. WHERE n-1 ★ ★ ★ ★

Previous row compare karne ke liye generally `LAG()` use hota hai.

Example

SQL

`SELECT`

`id,`

`marks,`

`LAG(marks)`

`OVER(`

`ORDER BY id`

`) previous_marks`

`FROM student;`



12. CURDATE() - INTERVAL

Current Date se calculation.

10 Days Add

 SQL

SELECT

CURDATE()+INTERVAL 10 DAY;

10 Days Minus

 SQL

SELECT

CURDATE()-INTERVAL 10 DAY;



14. TIMESTAMPDIFF()

Do dates ke beech ka difference nikalta hai.

Syntax

 SQL

TIMESTAMPDIFF(unit,start_date,end_date)



1 Month Add

SQL

SELECT

CURDATE()+INTERVAL 1 MONTH;

1 Month Minus

SQL

SELECT

CURDATE()-INTERVAL 1 MONTH;

1 Year Add

```
</> SQL  
  
SELECT  
  
CURDATE()+INTERVAL 1 YEAR;
```

1 Year Minus

```
</> SQL  
  
SELECT  
  
CURDATE()-INTERVAL 1 YEAR;
```

Table Example

Students added in last 30 days (assuming `admission_date` exists)

```
</> SQL  
  
SELECT *  
  
FROM student  
  
WHERE admission_date>=CURDATE()-INTERVAL 30 DAY;
```

13. CASE WHEN THEN ELSE 🌟

IF ELSE in SQL.

Syntax

```
</> SQL
```

```
CASE
```

```
WHEN condition THEN value
```

```
WHEN condition THEN value
```

```
ELSE value
```

```
END
```

```
</> SQL
```

```
SELECT
```

```
name,
```

```
marks,
```

```
CASE
```

```
WHEN marks >= 90 THEN 'A'
```

```
WHEN marks >= 80 THEN 'B'
```

```
WHEN marks >= 70 THEN 'C'
```

```
ELSE 'Fail'
```

```
END AS Grade
```

```
FROM student;
```

GPT ▾

Years

</> SQL

SELECT

TIMESTAMPDIFF(

YEAR,

'2004-05-10',

CURDATE())

);

Months

</> SQL

SELECT

TIMESTAMPDIFF(

MONTH,

'2024-01-01',

CURDATE())

);

Days

```
</> SQL  
  
SELECT  
  
TIMESTAMPDIFF(  
  
DAY,  
  
'2025-01-01',  
  
CURDATE()  
  
);
```

Student Age Example

Assume `dob` column exists.

```
</> SQL  
  
SELECT  
  
name,  
  
TIMESTAMPDIFF(  
  
YEAR,  
  
dob,  
  
CURDATE()  
  
)  
  
AS age  
  
FROM student;
```

```
-- 6. First 4 characters using SUBSTRING
```

```
SELECT SUBSTRING(name,1,4)
```

```
FROM student;
```

```
-- 7. Replace Indore with Delhi
```

```
SELECT REPLACE(city,'Indore','Delhi')
```

```
FROM student;
```

```
-- 8. Reverse every name
```

```
SELECT REVERSE(name)
```

```
FROM student;
```

```
-- 9. Remove extra spaces
```

```
SELECT TRIM(' Tarun ');
```

```
-- 10. Last 4 digits of phone
```

```
SELECT RIGHT(phone,4)
```

```
FROM student;
```

Function	Purpose	Example	Output
<code>ROUND()</code>	Nearest rounding	<code>ROUND(78.56)</code>	79
<code>CEIL()</code>	Hamesha upar	<code>CEIL(78.01)</code>	79
<code>FLOOR()</code>	Hamesha neeche	<code>FLOOR(78.99)</code>	78
<code>ABS()</code>	Absolute value	<code>ABS(-20)</code>	20
<code>MOD()</code>	Remainder	<code>MOD(10,3)</code>	1
<code>POWER()</code>	Power	<code>POWER(2,5)</code>	32
<code>SQRT()</code>	Square root	<code>SQRT(64)</code>	8
<code>RAND()</code>	Random number	<code>RAND()</code>	0.2345


```
-- 1. Average marks with 2 decimal places
SELECT ROUND(AVG(marks),2)
FROM student;

-- 2. Department-wise rounded average
SELECT dept_id,
ROUND(AVG(marks),2)
FROM student
GROUP BY dept_id;

-- 3. Ceiling example
SELECT CEIL(78.01);

-- 4. Floor example
SELECT FLOOR(78.99);

-- 5. Absolute difference from 80
SELECT name,
ABS(marks-80)
FROM student;
```

-- 6. Even ID students

```
SELECT *  
FROM student  
WHERE MOD(id,2)=0;
```

-- 7. Odd ID students

```
SELECT *  
FROM student  
WHERE MOD(id,2)=1;
```

-- 8. Marks square

```
SELECT name,  
POWER(marks,2)  
FROM student;
```

-- 9. Square root of marks

```
SELECT name,  
SQRT(marks)  
FROM student;
```

```
-- 10. Random 5 students
SELECT *
FROM student
ORDER BY RAND()
LIMIT 5;
```

Difference Table

Function	Purpose	Example	Output
NOW()	Date + Time	NOW()	2026-07-11 10:45
CURDATE()	Current Date	CURDATE()	2026-07-11
CURTIME()	Current Time	CURTIME()	10:45:30
DATEDIFF()	Days Difference	DATEDIFF('2026-07-11', '2026-07-01')	10
DATE_ADD()	Future Date	DATE_ADD(CURDATE(), INTERVAL 10 DAY)	2026-07-21
DATE_SUB()	Past Date	DATE_SUB(CURDATE(), INTERVAL 10 DAY)	2026-07-01
YEAR()	Year	YEAR(admission_date)	2023
MONTH()	Month	MONTH(admission_date)	7

```
-- 1
SELECT NOW();

-- 2
SELECT CURDATE();

-- 3
SELECT CURTIME();

-- 4
SELECT DATEDIFF(CURDATE(),admission_date)
FROM student;

-- 5
SELECT DATE_ADD(CURDATE(),INTERVAL 30 DAY);

-- 6
SELECT DATE_SUB(CURDATE(),INTERVAL 30 DAY);
```

```
-- 7
SELECT YEAR(admission_date)
FROM student;

-- 8
SELECT MONTH(admission_date)
FROM student;

-- 9
SELECT DAY(admission_date)
FROM student;

-- 10
SELECT *
FROM student
WHERE YEAR(admission_date)=2023;

-- 11
SELECT *
FROM student
WHERE MONTH(admission_date)=7;
```

SQL

```
-- 12
SELECT *
FROM student
WHERE DAY(admission_date)=15;

-- 13
SELECT name,
DATEDIFF(CURDATE(),admission_date)
FROM student;

-- 14
SELECT name,
DATE_ADD(admission_date,INTERVAL 6 MONTH)
FROM student;

-- 15
SELECT name,
DATE_SUB(admission_date,INTERVAL 1 YEAR)
FROM student;
```

Difference Table

Clause	Purpose	Example
DISTINCT	Duplicate values remove karta hai	SELECT DISTINCT city FROM stu
ORDER BY	Data ko sort karta hai	ORDER BY marks DESC
LIMIT	Rows ko limit karta hai	LIMIT 5

```
-- 1
SELECT DISTINCT city
FROM student;

-- 2
SELECT DISTINCT dept_id
FROM student;

-- 3
SELECT COUNT(DISTINCT city)
FROM student;

-- 4
SELECT *
FROM student
ORDER BY marks DESC;

-- 5
SELECT *
FROM student
ORDER BY marks;
```

```
-- 6
SELECT *
FROM student
ORDER BY name;

-- 7
SELECT *
FROM student
ORDER BY city,marks DESC;

-- 8
SELECT *
FROM student
ORDER BY dept_id,marks DESC;

-- 9
SELECT *
FROM student
LIMIT 5;
```



```
-- 10
SELECT *
FROM student
ORDER BY marks DESC
LIMIT 3;

-- 11
SELECT *
FROM student
ORDER BY marks
LIMIT 3;

-- 12
SELECT *
FROM student
ORDER BY age
LIMIT 1;

-- 13
SELECT DISTINCT city
FROM student
ORDER BY city;
```

```
-- 14
SELECT *
FROM student
LIMIT 5 OFFSET 5;

-- 15
SELECT *
FROM student
ORDER BY marks DESC
LIMIT 1 OFFSET 1;
```

📌 Interview tip

`DISTINCT` aur `GROUP BY` ko log aksar same samajhte hain, lekin ye alag hain:

- `DISTINCT` → Duplicate values hata kar unique values dikhata hai.
- `GROUP BY` → Records ko group karta hai, jisse `COUNT()`, `SUM()`, `AVG()` jaise aggregate functions use kiye ja sakte hain.

Example:

SQL

```
-- Sirf unique cities
SELECT DISTINCT city
FROM student;

-- Har city me kitne students hain
SELECT city, COUNT(*)
FROM student
GROUP BY city;
```



Subquery Type	Returns	Common Operators	Example
Scalar	1 value	= , > , <	<code>marks > (SELECT AVG(marks) FROM student)</code>
Multiple Row	Many rows	IN , ANY , ALL	<code>dept_id IN (SELECT dept_id FROM department)</code>
Nested	Subquery ke andar subquery	Any	Multi-level lookup
Correlated	Har outer row ke liye execute hoti hai	EXISTS , comparison	Department-wise topper
EXISTS	Kam se kam 1 matching row	EXISTS	Department with students
NOT EXISTS	Koi matching row nahi	NOT EXISTS	Department without students

Type	Return Value	Operator
Scalar	1 Row, 1 Column	=, >, <
Multiple Row	Multiple Rows	IN, ANY, ALL
Nested	Multiple Levels	Any
Correlated	Outer Query par depend	EXISTS
EXISTS	True/False	EXISTS
NOT EXISTS	True/False	NOT EXISTS

1. Scalar Subquery

Definition

Sirf 1 row aur 1 column return karti hai.

Example

 SQL

```
SELECT MAX(marks)
FROM student;
```

Output

97

Ye sirf ek value return kar rahi hai.

2. Multiple Row Subquery

Definition

Subquery ek se jyada rows return karti hai.

Iske saath use hote hain

- IN
- ANY
- ALL

Example

 SQL

```
SELECT dept_id  
FROM department  
WHERE block_name='A-Block';
```

3. Nested Subquery

Matlab

Subquery ke andar ek aur subquery.

Level 1



Level 2



Level 3

Example

Pehle department nikalo



Us department ka HOD nikalo



Us department ke students nikalo

</> SQL

```
SELECT *  
FROM student  
WHERE dept_id=  
(  
  SELECT dept_id  
  FROM department  
  WHERE hod_name=  
  (  
    SELECT hod_name  
    FROM department  
    WHERE dept_name='CSE'  
  )  
);
```

4. Correlated Subquery ★ ★ ★ ★ ★

Sabse important.

Definition

Outer query ki har row ke liye subquery dubara execute hoti hai.

Outer Row 1



Subquery



Outer query ki har row ke liye subquery dubara execute hoti hai.

Outer Row 1



Subquery



Outer Row 2



Subquery



Outer Row 3



Subquery



Department Wise Topper

SQL

```
SELECT *  
FROM student s  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

5. EXISTS

Definition

Agar subquery kam se kam ek row return kare → TRUE

Nahi kare → FALSE

Departments jinke students hain

 SQL

```
SELECT *  
FROM department d  
WHERE EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
);
```



6. NOT EXISTS

Reverse of EXISTS

Departments without students

 SQL

```
SELECT *  
FROM department d  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
);
```

Departments without Female Students

</> SQL

```
SELECT *  
FROM department d  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
  AND gender='Female'  
);
```

Departments without Topper (>95)

</> SQL

```
SELECT *  
FROM department d  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM student s  
  WHERE s.dept_id=d.dept_id  
  AND marks>95  
);
```

1 Highest Marks in Every Department

SQL

```
SELECT *  
FROM student s  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

2 Lowest Marks in Every Department

SQL

```
SELECT *  
FROM student s  
WHERE marks=  
(  
SELECT MIN(marks)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

4 Students Older Than Department Average

SQL

```
SELECT *  
FROM student s  
WHERE age >  
(  
SELECT AVG(age)  
FROM student  
WHERE dept_id=s.dept_id  
);
```

1/C/6a512cab-dcd0-03ee-a37b-da0b2300b11a

5 Department Having Maximum Students

SQL

```
SELECT dept_id, COUNT(*) total_students  
FROM student  
GROUP BY dept_id  
HAVING COUNT(*) =  
(  
SELECT MAX(cnt)  
FROM  
(  
SELECT COUNT(*) cnt  
FROM student  
GROUP BY dept_id  
) x  
);
```

6 Students in A-Block Departments

SQL

```
SELECT *  
FROM student  
WHERE dept_id IN  
(  
  SELECT dept_id  
  FROM department  
  WHERE block_name='A-Block'  
);
```


7 Students Not in A-Block

SQL

```
SELECT *  
FROM student  
WHERE dept_id NOT IN  
(  
  SELECT dept_id  
  FROM department  
  WHERE block_name='A-Block'  
);
```

8 Highest Marks Overall

SQL

```
SELECT *  
FROM student  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
);
```

9 Second Highest Marks

</> SQL

```
SELECT *  
FROM student  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE marks<  
(  
SELECT MAX(marks)  
FROM student  
)  
);
```

10 Third Highest Marks

SQL

```
SELECT *  
FROM student  
WHERE marks=  
(  
SELECT MAX(marks)  
FROM student  
WHERE marks<  
(  
SELECT MAX(marks)  
FROM student  
WHERE marks<  
(  
SELECT MAX(marks)  
FROM student  
)  
)  
);
```

3 Students Above Department Average

SQL

```
SELECT *  
FROM student s  
WHERE marks >  
(  
  SELECT AVG(marks)  
  FROM student  
  WHERE dept_id=s.dept_id  
);
```

1. Database Objects ★ ★ ★ ★

Database Object matlab database ke andar stored koi bhi object.

Types of Database Objects

Object	Purpose
Database	Data store karna
Table	Rows aur Columns me data
View	Virtual Table
Index	Fast Searching
Stored Procedure	Reusable SQL Program
Function	Value Return karta hai
Trigger	Automatically execute hota hai
Sequence	Auto Number Generate



1. Views ★ ★ ★ ★

Definition

View ek **Virtual Table** hoti hai.

Isme data physically store nahi hota.

Ye sirf SELECT query ka result save karti hai.

Student Table

|



SELECT Query

|



VIEW

Create View

SQL

```
CREATE VIEW student_view AS  
SELECT id,name,marks  
FROM student;
```

View Dekho

SQL

```
SELECT *  
FROM student_view;
```

Update View

SQL

```
CREATE OR REPLACE VIEW student_view AS  
SELECT id,name,marks,city  
FROM student;
```

Delete View

SQL

```
DROP VIEW student_view;
```

Index Kya Hai?

Index ek special data structure hai jo table me data ko jaldi search karne ke liye use hota hai.

Jaise kisi **book ke last me index** hota hai.

Agar tumhe "Database" topic dekhna hai to poori book nahi padhte.

Seedha index me page number dekhte ho.

Database me bhi wahi hota hai.

Purpose of Index

-  Fast Searching (WHERE)
-  Fast JOIN
-  Fast ORDER BY
-  Fast GROUP BY
-  Fast MIN() / MAX()
-  Performance Improve

Create Index

⌕ SQL

```
CREATE INDEX idx_name  
ON student(name);
```

Composite Index

⌕ SQL

```
CREATE INDEX idx_city_marks  
ON student(city,marks);
```

Show Index

⌕ SQL

```
SHOW INDEX  
FROM student;
```

Drop Index

⌕ SQL

```
DROP INDEX idx_name  
ON student;
```

Interview

Phone Search

SQL

```
CREATE INDEX idx_phone  
ON student(phone);
```

Composite Example

SQL

```
CREATE INDEX idx_city_dept  
ON student(city,dept_id);
```

Unique Index

Duplicate allow nahi karta.

SQL

```
CREATE UNIQUE INDEX idx_phone  
ON student(phone);
```

Show Index

SQL

```
SHOW INDEX  
FROM student;
```

Foreign Key Index

Foreign key par index **automatically** har database me nahi banta

Agar JOIN bahut hote hain, to manually banana achha hota hai.

SQL

```
CREATE INDEX idx_dept  
ON student(dept_id);
```

Types of Index

Type	Purpose
Primary Index	Primary Key
Unique Index	Duplicate not allowed
Composite Index	Multiple Columns
Clustered Index*	Data sorted physically
Non-Clustered Index*	Separate index structure
Full Text Index	Text Search

2. Stored Procedure ★★★★★

Definition

Stored Procedure ek **precompiled SQL program** hota hai jo database me save rehta hai.

Ek baar likho, baar-baar call karo.

Application



CALL Procedure



Procedure



Database



Create Procedure

```
</> SQL

DELIMITER $$

CREATE PROCEDURE GetStudents()
BEGIN
    SELECT * FROM student;
END $$

DELIMITER ;
```

Execute

```
</> SQL

CALL GetStudents();
```

Procedure with Parameter

SQL

DELIMITER \$\$

```
CREATE PROCEDURE StudentByCity
```

(

```
IN city_name VARCHAR(30)
```

)

BEGIN

SELECT *

FROM student

```
WHERE city=city_name;
```

END \$\$

DELIMITER ;

Execute

SQL

```
CALL StudentByCity('Indore');
```

Multiple Parameters

SQL

```
DELIMITER $$
```

```
CREATE PROCEDURE MarksGreater  
(  
  IN m INT,  
  IN c VARCHAR(30)  
)  
BEGIN
```

Multiple Parameters

SQL

```
DELIMITER $$
```

```
CREATE PROCEDURE MarksGreater
```

```
(
```

```
IN m INT,
```

```
IN c VARCHAR(30)
```

```
)
```

```
BEGIN
```

```
SELECT *
```

```
FROM student
```

```
WHERE marks>m
```

```
AND city=c;
```

```
END $$
```

```
DELIMITER ;
```

Call

SQL

```
CALL MarksGreater(80, 'Indore');
```

Drop Procedure

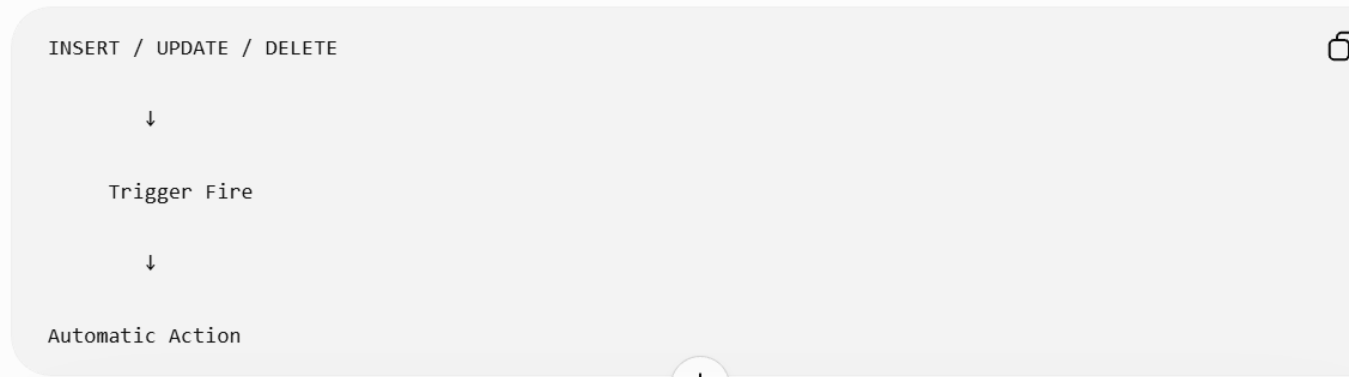
SQL

```
DROP PROCEDURE GetStudents;
```

Trigger Kya Hai?

Trigger ek database object hai jo kisi table par `INSERT` , `UPDATE` , ya `DELETE` hone par automatically execute hota hai.

Matlab trigger ko manually call nahi karna padta.



Types of Trigger

Trigger	Kab Chalta Hai
BEFORE INSERT	Insert se pehle
AFTER INSERT	Insert ke baad
BEFORE UPDATE	Update se pehle
AFTER UPDATE	Update ke baad
BEFORE DELETE	Delete se pehle
AFTER DELETE	Delete ke baad

```
</> SQL

DELIMITER $$

CREATE TRIGGER trigger_name

BEFORE/AFTER INSERT/UPDATE/DELETE

ON table_name

FOR EACH ROW

BEGIN

    SQL Statements;

END $$

DELIMITER ;
```


Log Table

SQL

```
CREATE TABLE student_log(  
  
  log_id INT AUTO_INCREMENT PRIMARY KEY,  
  
  student_id INT,  
  
  action VARCHAR(30),  
  
  action_time DATETIME  
  
);
```

1. AFTER INSERT Trigger ★ ★

Student insert hote hi log table me entry ho.

`</> SQL`

```
DELIMITER $$
```

```
CREATE TRIGGER trg_after_insert
```

```
AFTER INSERT
```

```
ON student
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO student_log
```

```
(student_id,action,action_time)
```

```
VALUES
```

```
</> SQL
```

```
VALUES
```

```
(NEW.id, 'Inserted', NOW());
```

```
END $$
```

```
DELIMITER ;
```

Test

```
</> SQL
```

```
INSERT INTO student
```

```
VALUES
```

```
(66, 'Raj', 20, 'Male', 'Indore', 2, 80, '9000000066', 101);
```



2. AFTER DELETE Trigger

Delete hone par log save karo.

</> SQL

```
DELIMITER $$
```

```
CREATE TRIGGER trg_after_delete
```

```
AFTER DELETE
```

```
ON student
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO student_log
```

```
(student_id,action,action_time)
```

```
VALUES
```

```
(student_id,action,action_time)
```

```
VALUES
```

```
(OLD.id,'Deleted',NOW());
```

```
END $$
```

```
DELIMITER ;
```

Test

```
</> SQL
```

```
DELETE
```

```
FROM student
```

```
WHERE id=66;
```

3. BEFORE UPDATE Trigger 🌟

Negative marks allow nahi karna.

`</>` SQL

```
DELIMITER $$
```

```
CREATE TRIGGER trg_before_update
```

```
BEFORE UPDATE
```

```
ON student
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.marks<0 THEN
```

```
SET NEW.marks=0;
```

```
END IF;
```

</> SQL

```
SET NEW.marks=-50;
```

```
END IF;
```

```
END $$
```

```
DELIMITER ;
```

Test

</> SQL

```
UPDATE student
```

```
SET marks=-50
```

```
WHERE id=2;
```

4. BEFORE INSERT Trigger

Marks 100 se jyada na ho.

`</> SQL`

```
DELIMITER $$
```

```
CREATE TRIGGER trg_before_insert
```

```
BEFORE INSERT
```

```
ON student
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.marks>100 THEN
```

```
SET NEW.marks=100;
```

```
END IF;
```


5. AFTER UPDATE Trigger

Marks update hone par log table me save karo.

```
</> SQL
```

```
DELIMITER $$
```

```
CREATE TRIGGER trg_after_update
```

```
AFTER UPDATE
```

```
ON student
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO student_log
```

```
(student_id,action,action_time)
```

```
VALUES
```

VALUES

```
(NEW.id, 'Marks Updated', NOW());
```

END \$\$

DELIMITER ;

OLD vs NEW ★★★★★

Keyword	Meaning
OLD	Update/Delete se pehle ka data
NEW	Insert/Update ke baad ka data

```
SHOW TRIGGERS;
```

Specific Trigger

SQL

```
SHOW CREATE TRIGGER trg_after_insert;
```

Trigger Delete

SQL

```
DROP TRIGGER trg_after_insert;
```

Multiple Trigger Example

INSERT Student

↓

BEFORE INSERT

↓

Student Insert

↓

AFTER INSERT

↓

Log Table Update

```
704512400 4000 0000 4070 440020000110
```

1. Salary Negative Nahi Honi Chahiye

SQL

```
IF NEW.salary<0 THEN
```

```
SET NEW.salary=0;
```

```
END IF;
```

2. Mobile Number 10 Digit

SQL

```
IF LENGTH(NEW.phone)!=10 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Invalid Phone Number';
```

```
END IF;
```

3. Marks Greater Than 100

</> SQL

```
IF NEW.marks>100 THEN
```

```
SET NEW.marks=100;
```

```
END IF;
```

4. Prevent Delete

</> SQL

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Delete Not Allowed';
```

Trigger	Stored Procedure
Automatically chalta hai	Manually <code>CALL</code> karna padta hai
Event par execute hota hai	Jab user call kare tab
INSERT/UPDATE/DELETE se trigger hota hai	Kisi bhi time execute ho sakta hai

Trigger vs Function

Trigger	Function
Auto execute	<code>SELECT</code> se call hoti hai
Event based	Value return karti hai
Return value nahi hoti	Return value mandatory hoti hai

Function Kya Hai?

Function SQL ka ek database object hai jo input (parameters) leta hai, kuch processing karta hai aur hamesha ek value return karta hai.

Rule: SQL Function ka **return value mandatory** hota hai.

Real Life Example

Calculator

Number = 25

↓

`SQRT()`

↓

5



Purpose of Function

- ☒ Calculation karna
- ☒ Reusable code likhna
- ☒ Business logic implement karna
- ☒ Data formatting
- ☒ Validation
- ☒ Code duplication kam karna

</> SQL

```
DELIMITER $$
```

```
CREATE FUNCTION function_name(parameter datatype)
```

```
RETURNS datatype
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    SQL Statements
```

```
    RETURN value;
```

```
END $$
```

```
DELIMITER ;
```


Example 1 (Bonus Function)

`</> SQL`

`DELIMITER $$`

`CREATE FUNCTION Bonus(m INT)`

`RETURNS INT`

`DETERMINISTIC`

`BEGIN`

`RETURN m+5;`

`END $$`

`DELIMITER ;`

Example 2 (Pass/Fail Function)

SQL

```
DELIMITER $$
```

```
CREATE FUNCTION Result(m INT)
```

```
RETURNS VARCHAR(10)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
IF m >= 40 THEN
```

```
RETURN 'PASS';
```

```
ELSE
```

```
RETURN 'FAIL';
```

```
END IF;
```

Table ke saath

`</> SQL`

`SELECT`

`name,`

`marks,`

`Result(marks)`

`FROM student;`

Example 3 (Grade Function)

SQL

```
DELIMITER $$
```

```
CREATE FUNCTION Grade(m INT)
```

```
RETURNS CHAR(1)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
IF m >= 90 THEN
```

```
RETURN 'A';
```

```
ELSEIF m >= 80 THEN
```

```
RETURN 'B';
```

```
RETURN 'B';

ELSEIF m>=70 THEN

RETURN 'C';

ELSE

RETURN 'D';

END IF;

END $$

DELIMITER ;
```

Use

```
</> SQL

SELECT

name,

marks,

Grade(marks)

FROM student;
```

Example 4 (Age Category)

```
</> SQL

DELIMITER $$

CREATE FUNCTION AgeCategory(a INT)

RETURNS VARCHAR(20)

DETERMINISTIC

BEGIN

IF a<18 THEN

RETURN 'Minor';

ELSE

RETURN 'Adult';

END IF;
```

Show Functions

`</> SQL`

```
SHOW FUNCTION STATUS;
```

Delete Function

`</> SQL`

```
DROP FUNCTION Bonus;
```

Function vs Stored Procedure

Function

Value return karna mandatory

`SELECT` se call hoti hai

Calculation ke liye

DML (INSERT/UPDATE/DELETE) ka use generally nahi karte

Stored Procedure

Return optional

`CALL` se execute hoti hai

Business operations ke liye

DML operations kar sakti hai

Function vs Trigger

Function	Trigger
User call karta hai	Automatically execute hota hai
Value return karti hai	Return value nahi hoti
<code>SELECT</code> me use hoti hai	Event (<code>INSERT/UPDATE/DELETE</code>) par chalti hai

Window Function Kya Hai?

Window Function rows par calculation karti hai bina rows ko collapse kiye.

Normal Aggregate Function

`</>` SQL

```
SELECT dept_id,AVG(marks)
FROM student
GROUP BY dept_id;
```

Window Function

`</>` SQL

```
SELECT
name,
marks,
AVG(marks) OVER()
FROM student;
```


Purpose

-  Ranking
-  Running Total
-  Department Wise Topper
-  Previous Row Compare
-  Next Row Compare
-  Moving Average
-  Salary Comparison
-  Percentage Calculation

Syntax

```
</> SQL  
  
Function()  
  
OVER(  
  
PARTITION BY column  
  
ORDER BY column  
  
)
```

OVER()

Ye batata hai calculation kis rows ke group (window) par hogi.

SQL

```
SELECT  
name,  
marks,  
AVG(marks)  
OVER()  
FROM student;
```

PARTITION BY

Rows ko groups me divide karta hai.

Student

↓

Department Wise Groups

↓

Calculation

Example

Example

SQL

```
SELECT
name,
dept_id,
marks,
AVG(marks)
OVER(PARTITION BY dept_id)
FROM student;
```

ORDER BY

Ranking ya sequence ke liye.

SQL

```
SELECT
name,
marks,
ROW_NUMBER()
OVER(ORDER BY marks DESC)
FROM student;
```

Types of Window Functions

Function	Purpose
ROW_NUMBER()	Unique Rank
RANK()	Same Rank, Gap
DENSE_RANK()	Same Rank, No Gap
NTILE()	Groups me divide
LEAD()	Next Row
LAG()	Previous Row
FIRST_VALUE()	First Value
LAST_VALUE()	Last Value

Sql

ipit.com/c/6a512cab-dcd0-83ee-a37b-da062380811a

LAG()	Previous Row
FIRST_VALUE()	First Value
LAST_VALUE()	Last Value
SUM() OVER()	Running Total
AVG() OVER()	Running Average
COUNT() OVER()	Running Count
MAX() OVER()	Maximum
MIN() OVER()	Minimum

1 ROW_NUMBER()

Har row ko unique number deta hai.

</> SQL

```
SELECT
name,
marks,
ROW_NUMBER()
OVER(ORDER BY marks DESC) AS rn
FROM student;
```

Department Wise

</> SQL

```
SELECT
name,
dept_id,
marks,
ROW_NUMBER()
OVER(
PARTITION BY dept_id
ORDER BY marks DESC
) rn
FROM student;
```

Window Function vs GROUP BY

GROUP BY

Rows combine ho jaati hain

Aggregate result deta hai

Detail rows nahi milti

Ranking possible nahi

Window Function

Rows same rehti hain

Aggregate + original rows dono deta hai

Detail rows milti hain

Ranking possible hai

RANK() Function in SQL ★ ★ ★ ★ ★ (Window Function)

RANK() Kya Hai?

RANK() ek Window Function hai jo rows ko rank (position) deta hai. Agar do ya zyada rows ki value same ho, to unhe same rank milti hai aur next rank skip ho jati hai.

Example:

Marks

97
97
95
90



Rank

97 → Rank 1



Syntax

SQL

```
SELECT  
column1,  
column2,  
RANK() OVER(  
ORDER BY column DESC  
) AS rank  
FROM table_name;
```


Example 1 (Overall Ranking)

```
</> SQL

SELECT
  id,
  name,
  marks,
  RANK() OVER(
    ORDER BY marks DESC
  ) AS student_rank
FROM student;
```

Example 3 (Department Wise Ranking)

Har department ke andar ranking.

```
</> SQL

SELECT
  name,
  dept_id,
  marks,
  RANK() OVER(
    PARTITION BY dept_id
    ORDER BY marks DESC
  ) AS dept_rank
FROM student;
```

Example 4 (Topper of Each Department)

```
</> SQL

SELECT *
FROM
(
    SELECT *,
        RANK() OVER(
            PARTITION BY dept_id
            ORDER BY marks DESC
        ) AS rnk
    FROM student
) t
WHERE rnk = 1;
```

Example 5 (Top 3 Students of Every Department)

```
</> SQL

SELECT *
FROM
(
    SELECT *,
        RANK() OVER(
            PARTITION BY dept_id
            ORDER BY marks DESC
        ) AS rnk
    FROM student
) t
WHERE rnk <= 3;
```

Example 6 (Second Highest Marks)

```
</> SQL

SELECT *
FROM
(
    SELECT *,
        RANK() OVER(
            ORDER BY marks DESC
        ) AS rnk
    FROM student
) t
WHERE rnk = 2;
```

Note: Agar highest marks tie hain (do students ke 97 marks), to `RANK()` me next rank 3 hogi. Isliye second highest unique marks nikalne ke liye aksar `DENSE_RANK()` zyada suitable hota hai.

MAX() with GROUP BY

Har department ka highest marks.

```
</> SQL

SELECT
dept_id,
MAX(marks) AS highest_marks
FROM student
GROUP BY dept_id;
```

MAX() with HAVING

Sirf un departments ko dikhao jinke highest marks 90 se jyada hain.

</> SQL

```
SELECT
dept_id,
MAX(marks) AS highest_marks
FROM student
GROUP BY dept_id
HAVING MAX(marks) > 90;
```

MAX() with JOIN

Department Name ke saath Highest Marks.

</> SQL

```
SELECT
d.dept_name,
MAX(s.marks) AS highest_marks
FROM student s
JOIN department d
ON s.dept_id = d.dept_id
GROUP BY d.dept_name;
```

MAX() with Subquery

Highest marks wale student ki details.

</> SQL

```
SELECT *  
FROM student  
WHERE marks = (  
    SELECT MAX(marks)  
    FROM student  
);
```

Second Highest Marks

</> SQL

```
SELECT MAX(marks)  
FROM student  
WHERE marks < (  
    SELECT MAX(marks)  
    FROM student  
);
```

Third Highest Marks

SQL

```
SELECT MAX(marks)
FROM student
WHERE marks < (
    SELECT MAX(marks)
    FROM student
    WHERE marks < (
        SELECT MAX(marks)
        FROM student
    )
);
```

Highest Marks in Each City

</> SQL

```
SELECT
city,
MAX(marks) AS highest_marks
FROM student
GROUP BY city;
```

Highest Marks in Each Department (with Student Name)

</> SQL

```
SELECT s.*
FROM student s
WHERE marks = (
    SELECT MAX(marks)
    FROM student
    WHERE dept_id = s.dept_id
);
```



MAX() as Window Function

Har student ke saath uske department ka maximum marks.

</> SQL

```
SELECT
name,
dept_id,
marks,
MAX(marks) OVER(PARTITION BY dept_id) AS dept_max
FROM student;
```

CTE Kya Hai?

CTE (Common Table Expression) ek temporary result set hota hai jo query ke execution ke dauran hi exist karta hai.

- Ye permanent table **nahi** hota.
- Sirf us query ke liye available hota hai.
- Query khatam → CTE bhi khatam.

Socho ki CTE ek **temporary virtual table** hai.

Purpose of CTE

- ☒ Complex query ko simple banana
- ☒ Readability improve karna
- ☒ Subquery ki jagah use karna
- ☒ Recursive queries likhna
- ☒ Multiple baar same result use karna
- ☒ Ranking aur Window Functions ke saath use ... a

Example 1 – Basic CTE

80 se jyada marks wale students.

 SQL

```
WITH topper AS
(
    SELECT *
    FROM student
    WHERE marks > 80
)

SELECT *
FROM topper;
```


Example 2 – Aggregate Function

Average marks nikalna.

</> SQL

```
WITH avg_marks AS
(
    SELECT AVG(marks) AS avg_mark
    FROM student
)

SELECT *
FROM avg_marks;
```

Example 3 – Students Above Average

</> SQL

```
WITH avg_table AS
(
    SELECT AVG(marks) AS avg_marks
    FROM student
)

SELECT *
FROM student
WHERE marks >
(
    SELECT avg_marks
    FROM avg_table
);
```

Example 4 – Department Wise Average

</> SQL

```
WITH dept_avg AS
(
    SELECT
        dept_id,
        AVG(marks) AS average_marks
    FROM student
    GROUP BY dept_id
)

SELECT *
FROM dept_avg;
```



Example 5 – JOIN with CTE

</> SQL

```
WITH dept_avg AS
(
    SELECT
        dept_id,
        AVG(marks) avg_marks
    FROM student
    GROUP BY dept_id
)

SELECT
    d.dept_name,
    a.avg_marks
FROM dept_avg a
JOIN department d
ON a.dept_id=d.dept_id;
```

Example 6 – Multiple CTE

```
</> SQL

WITH avg_marks AS
(
    SELECT AVG(marks) avg_mark
    FROM student
),

max_marks AS
(
    SELECT MAX(marks) max_mark
    FROM student
)

SELECT
avg_mark,
max_mark
FROM avg_marks,max_marks;
```

Example 7 – Window Function + CTE

Department Wise Rank

SQL

```
WITH ranking AS  
(  
  SELECT  
  
    name,  
  
    dept_id,  
  
    marks,  
  
    RANK() OVER(  
  
      PARTITION BY dept_id  
  
      ORDER BY marks DESC  
  
    ) rnk
```



Example 8 – Highest Marks Student

SQL

```
WITH highest AS
(
  SELECT MAX(marks) highest_marks

  FROM student
)

SELECT *

FROM student

WHERE marks=

(
  SELECT highest_marks

  FROM highest
)
```



Example 9 – Lowest Marks

SQL

```
WITH lowest AS
(
  SELECT MIN(marks) m

  FROM student
)

SELECT *

FROM student

WHERE marks=

(

  SELECT m

  FROM lowest

):
```

Recursive CTE Kya Hai?

Recursive CTE ek special type ka CTE hai jo khud ko (itself) baar-baar call karta hai jab tak condition false na ho jaye.

Ye recursion (loop) ki tarah kaam karta hai.

Use Cases:

- Employee Hierarchy (CEO → Manager → Employee)
- Folder Structure
- Category Tree
- Family Tree
- 1 se N tak numbers generate karna
- Graph Traversal

</> SQL

```
WITH RECURSIVE cte_name AS
(
    -- Anchor Query (Starting Point)

    SELECT ...

    UNION ALL

    -- Recursive Query

    SELECT ...
    FROM cte_name
    WHERE condition
)

SELECT *
FROM cte_name;
```

Recursive CTE ke 2 parts hote hain:

1. **Anchor Query** → Starting value
2. **Recursive Query** → Repeat hoti hai

Recursive CTE vs Normal CTE

Normal CTE	Recursive CTE
Sirf ek baar execute hota hai	Baar-baar execute hota hai
Self reference nahi hota	Khud ko reference karta hai
Complex query simplify karta hai	Hierarchy aur sequence generate karta hai
WITH	WITH RECURSIVE

Example 1: 1 se 10 Print Karo

</> SQL

```
WITH RECURSIVE numbers AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n + 1
    FROM numbers
    WHERE n < 10
)

SELECT *
FROM numbers;
```

Example 3: Even Numbers

SQL

```
WITH RECURSIVE even_no AS
(
    SELECT 2 AS n

    UNION ALL

    SELECT n+2
    FROM even_no
    WHERE n<20
)

SELECT *
FROM even_no;
```

Example 4: Odd Numbers

SQL

```
WITH RECURSIVE odd_no AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n+2
    FROM odd_no
    WHERE n<19
)

SELECT *
FROM odd_no;
```

Example 5: Multiplication Table of 5

SQL

```
WITH RECURSIVE table5 AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n+1
    FROM table5
    WHERE n<10
)

SELECT
n,
5*n AS result
FROM table5;
```

Example 6: Factorial

SQL

```
WITH RECURSIVE fact AS
(
    SELECT
        1 AS n,
        1 AS factorial

    UNION ALL

    SELECT
        n+1,
        factorial*(n+1)
    FROM fact
    WHERE n<5
)

SELECT *
FROM fact;
```

```
</> SQL
```

```
WITH RECURSIVE hierarchy AS
(
    SELECT
        emp_id,
        emp_name,
        manager_id
    FROM employee
    WHERE manager_id IS NULL

    UNION ALL

    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id
    FROM employee e
    JOIN hierarchy h
        ON e.manager_id = h.emp_id
)

SELECT *
FROM hierarchy;
```

Example 8: Countdown

</> SQL

```
WITH RECURSIVE countdown AS
(
    SELECT 10 AS n

    UNION ALL

    SELECT n-1
    FROM countdown
    WHERE n>1
)

SELECT *
FROM countdown;
```

Output

Example 9: Sum 1 to 10

SQL

```
WITH RECURSIVE numbers AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n+1
    FROM numbers
    WHERE n<10
)

SELECT SUM(n)
FROM numbers;
```